

Subset Sum Backtracking

1 Problem Definition

Subset Sum Problem

Given positive integers

$$w[1], w[2], \dots, w[n]$$

and a target value m , find every subset whose sum is exactly m .

A binary solution vector is used:

$$x[i] \in \{0, 1\}.$$

The meaning is

$$x[i] = \begin{cases} 1, & \text{include } w[i], \\ 0, & \text{exclude } w[i]. \end{cases}$$

Solution Equation

A subset is a solution exactly when

$$\sum_{i=1}^n w[i]x[i] = m.$$

2 Example

Use

$$n = 6, \quad m = 30,$$

$$w[1 : 6] = \{5, 10, 12, 13, 15, 18\}.$$

The weights are already sorted in increasing order:

$$5 < 10 < 12 < 13 < 15 < 18.$$

The total sum is

$$5 + 10 + 12 + 13 + 15 + 18 = 73.$$

Root State

The root of the search tree is written as

$$0 \mid 73.$$

The left value is the current selected sum s . The right value is the remaining available sum r .

3 State Representation

At any node, write

$$s \mid r.$$

In these notes, k means the next item to decide. Therefore, before deciding $w[k]$,

$$s = \sum_{i=1}^{k-1} w[i]x[i], \quad r = \sum_{i=k}^n w[i].$$

So r includes the current item $w[k]$ and all later items.

Symbol	Meaning
s	Current sum of the selected elements.
k	Next item index to decide.
r	Sum of items still available from $w[k]$ through $w[n]$.
m	Target sum.
$x[1..n]$	Binary include/exclude solution vector.

At level k , the algorithm has two choices:

$$\boxed{x[k] = 1 \quad \text{include } w[k]} \quad \text{or} \quad \boxed{x[k] = 0 \quad \text{exclude } w[k]}.$$

State Update

If $w[k]$ is included:

$$s_{\text{new}} = s + w[k], \quad r_{\text{new}} = r - w[k].$$

If $w[k]$ is excluded:

$$s_{\text{new}} = s, \quad r_{\text{new}} = r - w[k].$$

4 Bounding Conditions

Backtracking avoids branches that cannot possibly reach the target.

4.1 Overshoot Bound

Since all weights are positive, once the current sum exceeds the target, adding more elements can never fix it:

$$\boxed{s > m \implies \text{prune.}}$$

The next-inclusion bound is

$$\boxed{s + w[k] \leq m.}$$

This says that including the next item $w[k]$ should not already overshoot the target.

4.2 Remaining-Sum Bound

Even if no overshoot occurs, the branch is useless if the remaining values are too small to reach m . If all remaining available values together cannot reach the target, prune:

$$\boxed{s + \sum_{i=k}^n w[i] < m \implies \text{prune.}}$$

Equivalently, a branch may continue only when

$$s + \sum_{i=k}^n w[i] \geq m.$$

Equality Must Be Allowed

The mathematically correct continuation condition is $\geq m$, not $> m$, because equality means the target is still exactly reachable.

5 Promising Function

For a clean mathematical version, suppose $w[1], \dots, w[n]$ are sorted positive integers. At a node before deciding index k , let

$$s = \sum_{i=1}^{k-1} w[i]x[i], \quad R_k = \sum_{i=k}^n w[i].$$

Promising Test

The node is promising when

$$s \leq m \quad \text{and} \quad s + R_k \geq m.$$

The first condition prevents overshoot. The second condition guarantees that enough remaining weight exists to possibly reach the target.

Child Tests

From a promising node before deciding $w[k]$:

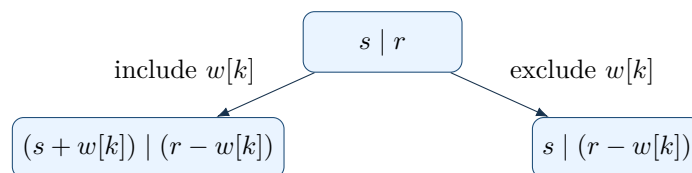
$$\text{include } w[k] \text{ only if } s + w[k] \leq m,$$

and

$$\text{exclude } w[k] \text{ only if } s + (R_k - w[k]) \geq m.$$

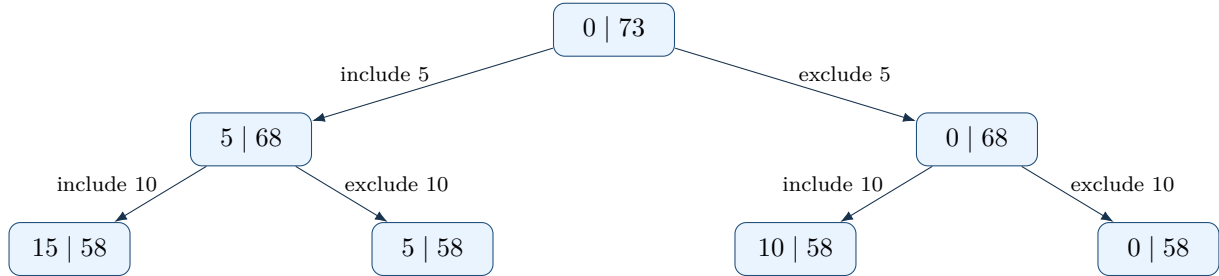
6 Include/Exclude Tree

The state-space tree is binary. Each level decides whether one weight is included or excluded.



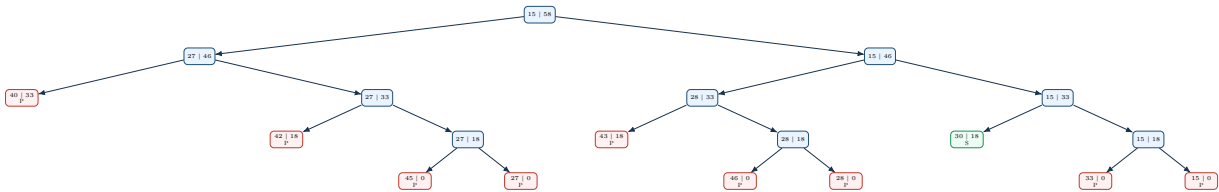
7 Complete Example Tree in Parts

The full tree is too wide for one readable diagram, so it is split into smaller parts. In every subtree, the left child means **include the next item**, and the right child means **exclude the next item**. Green nodes marked S are solutions, and red nodes marked P are pruned.



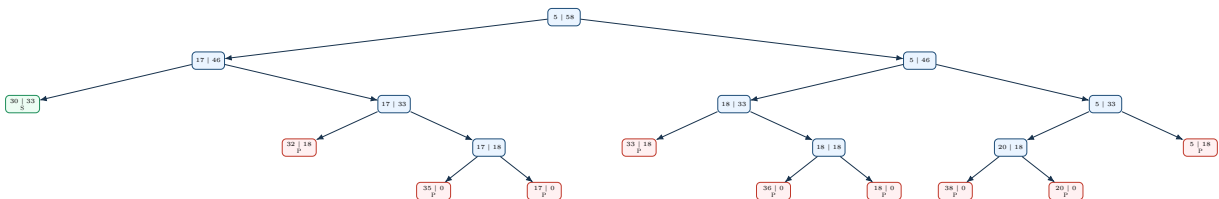
Part A: Include 5, Include 10

This subtree starts from 15 | 58, so the selected sum already contains 5 + 10. The remaining decisions are for 12, 13, 15, 18.



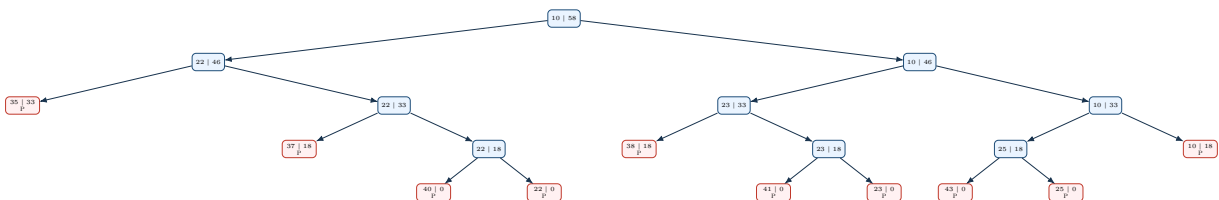
Part B: Include 5, Exclude 10

This subtree starts from 5 | 58. The remaining decisions are for 12, 13, 15, 18.



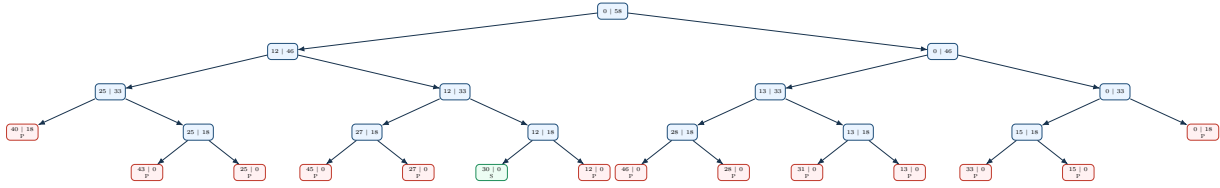
Part C: Exclude 5, Include 10

This subtree starts from 10 | 58. The remaining decisions are for 12, 13, 15, 18.



Part D: Exclude 5, Exclude 10

This subtree starts from $0 \mid 58$. The remaining decisions are for 12, 13, 15, 18.



All Solution Leaves

The complete tree has exactly three solution leaves:

$\{5, 10, 15\}$, $\{5, 12, 13\}$, $\{12, 18\}$.

8 Checking One Solution Path

Problem

Find a subset of

$\{5, 10, 12, 13, 15, 18\}$

whose sum is 30.

Follow the branch

$5 \rightarrow 10 \rightarrow \text{exclude } 12 \rightarrow \text{exclude } 13 \rightarrow 15$.

The selected sum is

$$5 + 10 + 15 = 30.$$

Thus the binary vector is

$$x = [1, 1, 0, 0, 1, 0].$$

Check the solution equation:

$$\begin{aligned} \sum_{i=1}^6 w[i]x[i] &= 5(1) + 10(1) + 12(0) + 13(0) + 15(1) + 18(0). \\ &= 5 + 10 + 0 + 0 + 15 + 0 = 30. \end{aligned}$$

Checked Path

This branch gives the valid subset

$\{5, 10, 15\}$.

The other solution leaves are already listed at the end of the complete tree.

9 Pseudocode

The pseudocode uses

$\text{SubsetSum}(k, s, r)$,

where k is the next item to decide, s is the current selected sum, and r is the remaining sum from $w[k]$ onward. Initially,

$$\text{SubsetSum}(1, 0, \sum_{i=1}^n w[i]).$$

Subset Sum Backtracking

```
SubsetSum(k, s, r):
    if s == m:
        print the selected set
        return

    if k > n:
        return

    if s > m or s + r < m:
        return

    x[k] = 1
    if s + w[k] <= m:
        SubsetSum(k + 1, s + w[k], r - w[k])

    x[k] = 0
    if s + r - w[k] >= m:
        SubsetSum(k + 1, s, r - w[k])
```

Why This Version Is Cleaner

Because k is the next item to decide, the algorithm tests $w[k]$, not $w[k + 1]$. The boundary case is handled directly by

$$k > n.$$

10 Correctness Intuition

The include/exclude tree contains every possible subset, because each element receives exactly one binary decision:

$$x[i] = 1 \quad \text{or} \quad x[i] = 0.$$

The algorithm prints only when the selected sum equals m , so every printed subset is valid. The bounds prune only when the sum is already too large or the remaining values are insufficient, so no reachable solution is discarded.

Conclusion

Backtracking searches the subset tree, but the bounds remove branches that cannot satisfy

$$\sum_{i=1}^n w[i]x[i] = m.$$

11 Complexity

Without pruning, there are two choices for each of n elements:

$$2^n$$

possible binary vectors.

Therefore, the worst-case time remains exponential:

$$\mathcal{O}(2^n).$$

The recursion depth is at most n , and the solution vector has length n , so the auxiliary space is

$$\mathcal{O}(n).$$

Practical Effect

Sorting and bounding do not remove the exponential worst case, but they often reduce the number of recursive calls dramatically.

12 Final Review

Subset Sum Essentials

$x[i] \in \{0, 1\}$ records exclude/include decisions.
 $\sum_{i=1}^n w[i]x[i] = m$ defines a solution.
 $s > m$ means overshoot, so prune.
 $s + R_k < m$ means insufficient remaining sum, so prune.
After one solution is printed, continue searching for others.